Cours Modélisation et vérification des systèmes informatiques
Exercices
Modélisation d'algorithmes en PlusCal (I)
par Dominique Méry
16 octobre 2026

Exercice 1 *appex4...*
Nous allons utiliser la fonctionnalité de traduction d'un algorithme PlusCal en un module TLA pour vérifier des algorithmes. Pour chaque question, on écrira un module TLA contenant une expressiond e l'algorithme puis on traduira par un module et ensuite on analysera le module obtenu par rapport à la correction partielle et l'absence d'erreurs à l'exécution.

Question 1.1 *appex4_1_1.tla*

$\ell_1 : x = 10 \ \wedge \ y = z{+}x \ \wedge z = 2{\cdot}x$
$y := z{+}x$
$\ell_2 : x = 10 \ \wedge \ y = x{+}2{\cdot}10$

Question 1.2 *On suppose que p est un nombre premier :*

$\ell_1 : x = 2^p \ \wedge \ y = 2^{p+1} \ \wedge \ x{\cdot}y = 2^{2\cdot p+1}$
$x := y{+}x{+}2^x$
$\ell_2 : x = 5{\cdot}2^p \ \wedge \ y = 2^{p+1}$

Question 1.3

$\ell_1 : x = 1 \ \wedge \ y = 12$
$x := 2{\cdot}y$
$\ell_2 : x = 1 \ \wedge \ y = 24$

Question 1.4

$\ell_1 : x = 11 \ \wedge \ y = 13$
$z := x; x := y; y := z;$
$\ell_2 : x = 26/2 \ \wedge \ y = 33/3$

Exercice 2 *Annoter chaque algorithme et le traduire avec la traduction de PlusCal en TLA$^+$.*

Question 2.1 *pluscal_loop*

Listing 1 – Fragment

```
          variables x=x0;
{
 while (0<x)
  {
     x := x - 1;
   };
}
}
```

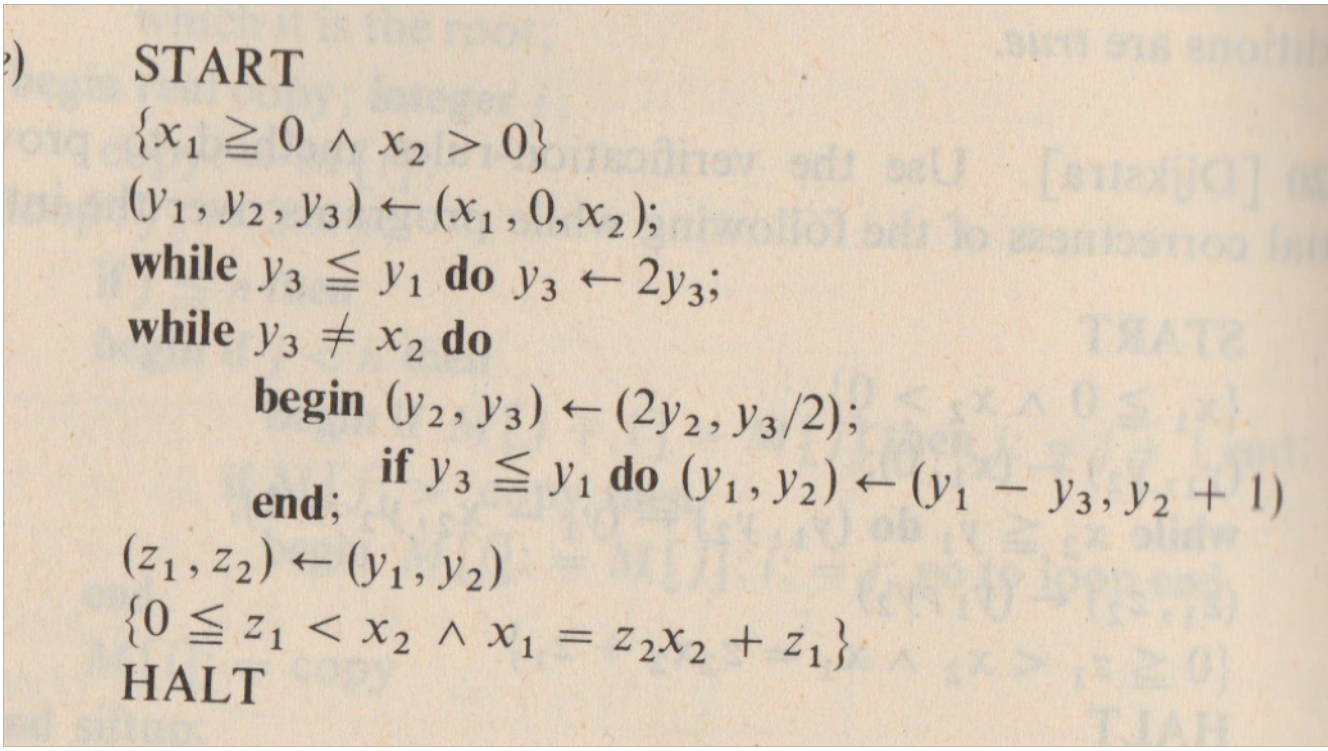
Question 2.2 *pluscal_max*

Listing 2 – Fragment

```
  variables x=a,y = b,z;
{
```

```
 if ( x < y ) {
  z:=y;
  }
else
{
 z:= x;
 };
 }
}
```

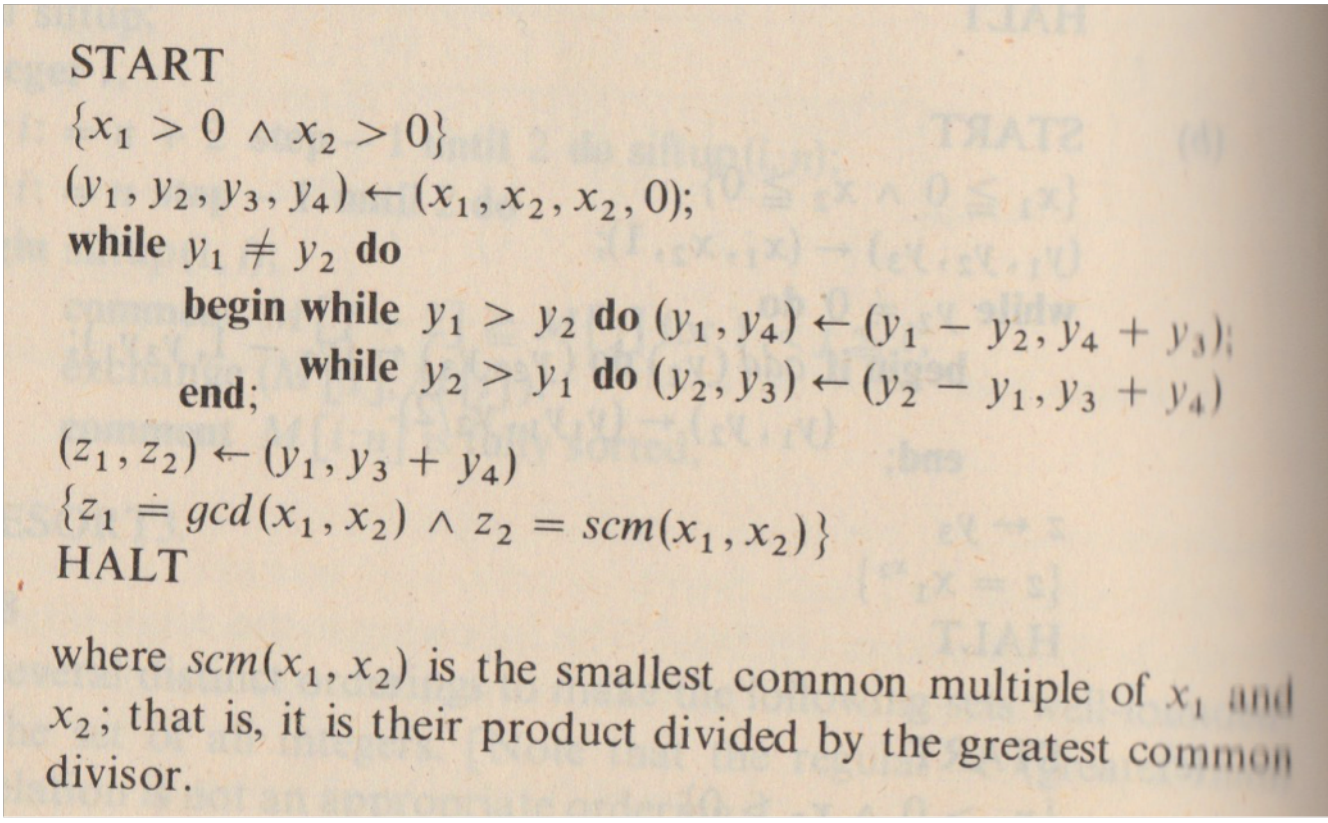
Exercice 3 *appex4_3.tla*
On considère l'algorithme suivant :



Question 3.1 *Montrer que cet algorithme est aprtiellement correct par rapport à sa précondition et à sa postcondition qu'il faudra énoncer. Pour cela, on traduira cet algorithme sous forme d'un module à partir du langage PlusCal.*

Question 3.2 *Montrer qu'il est sans erreur à l'exécution.*

Exercice 4 *appex4_4.tla*
On considère l'algorithme suivant :



START
$\{x_1 > 0 \wedge x_2 > 0\}$
$(y_1, y_2, y_3, y_4) \leftarrow (x_1, x_2, x_2, 0);$
while $y_1 \neq y_2$ **do**
    begin while $y_1 > y_2$ **do** $(y_1, y_4) \leftarrow (y_1 - y_2, y_4 + y_3);$
    end; **while** $y_2 > y_1$ **do** $(y_2, y_3) \leftarrow (y_2 - y_1, y_3 + y_4)$
$(z_1, z_2) \leftarrow (y_1, y_3 + y_4)$
$\{z_1 = gcd(x_1, x_2) \wedge z_2 = scm(x_1, x_2)\}$
HALT

where $scm(x_1, x_2)$ is the smallest common multiple of x_1 and x_2; that is, it is their product divided by the greatest common divisor.

Question 4.1 *Montrer que cet algorithme est aprtiellement correct par rapport à sa précondition et à sa postcondition qu'il faudra énoncer. Pour cela, on traduira cet algorithme sous forme d'un module à partir du lanage PlusCal.*

Question 4.2 *Montrer qu'il est sans erreur à l'exécution.*